

Exercise Mean-Shift

Ziyu Ma

Implementation:

```
def distance(x, X):
    return np.linalg.norm(X - x, axis=1)

def gaussian(dist, bandwidth):
    return np.exp(- 0.5 * (dist / bandwidth) ** 2)

def update_point(weight, X):
    return np.sum(weight[:, np.newaxis] * X, axis=0) / np.sum(weight)

def meanshift_step(X, bandwidth=3):
    X_ = X.copy()
    for i, x in enumerate(X):
        dist = distance(x, X)
        weight = gaussian(dist, bandwidth)
        X_[i] = update_point(weight, X)
    return X_

def meanshift(X):
    for _ in range(20):
        X = meanshift_step(X)
    return X
```

General: The *meanshift_step* function is called by the *meanshift* function 20 rounds. In the *meanshift_step* function, for each point among *X*, I calculate the distance of it with all other points using the *distance* function, weigh the distance with a *gaussian* function, the weights are then used by the *update_point* function to update the point position into a new *X_* numpy array.

For the *distance* function: The expression *X-x* computes the element-wise difference between each point in the array *X* and the specific point *x*. The *np.linalg.norm* function is then used to calculate the Euclidean norm along axis 1.

For the *gaussian* function: not much to say, just use the gaussian formula.

For the *update_point* function: first add a new axis to *weight*, such that elementwise multiplication with *X* can be done. The weighed positions are then added together using *np.sum*. *np.sum(weight)* is used to normalize the weights.

Formula used:

$$f(x) = \sum_{n=1}^N \frac{K'(\|\frac{x-x_n}{\sigma}\|^2)}{\sum_{n'=1}^N K'(\|\frac{x-x_{n'}}{\sigma}\|^2)} x_n$$

Result:

Result pictures are on next page. As the bandwidth increases, the number of centroids and labels decreases. So when dividing by the same number 4 in *centroids*, *labels = np.unique((X/4).round(), return_inverse=True, axis=0)*, the pictures are labels by less colors with bigger bandwidth. The labeling is also with less detail.

With small bandwidth 1 and sometimes also 2.5 the code is not runnable, the reason is there is so many centroids that the labels are much more than the colors provided. Divide *X* by a larger number solves the problem, e.g. by 30.

```
(meanshift) PS D:\2023HS\Computer Vision\Image Segmentation\mean-shift> python mean-shift.py
Elapsed time for mean-shift: 425.33514428138733
Traceback (most recent call last):
  File "D:\2023HS\Computer Vision\Image Segmentation\mean-shift\mean-shift.py", line 51, in <module>
    result_image = colors[labels].reshape(shape)
IndexError: index 278 is out of bounds for axis 0 with size 24
```



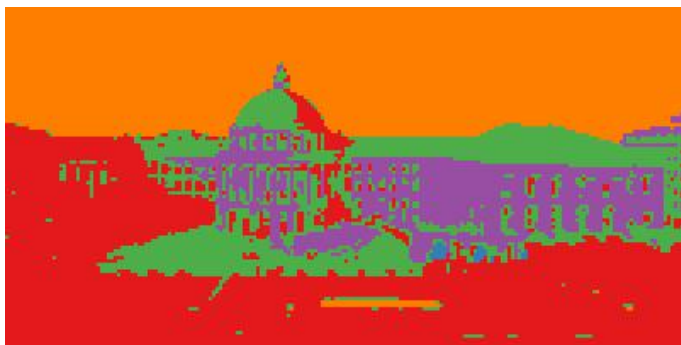
bandwidth = 1, divide by 30



bandwidth = 2.5, divide by 4



bandwidth = 3, divide by 4



bandwidth = 5, divide by 4



bandwidth = 7, divide by 4